TECHNICAL DOCUMENTATION

RAG-Based Customer Support Assistant (with LangGraph & HITL)

---

1. Introduction

1.1 What is RAG?

Retrieval-Augmented Generation (RAG) is an AI approach that combines two key processes:

- Retrieval: Fetching relevant information from a knowledge base
- Generation: Producing responses using retrieved data

Unlike traditional models, RAG uses external documents, making responses more accurate and context-aware.

1.2 Why RAG is Needed

Traditional AI systems:

- May generate incorrect answers
- Lack domain-specific knowledge
- Cannot dynamically update data

RAG solves this by:

- Using real documents
- Retrieving relevant context
- Generating grounded responses

1.3 Use Case Overview

This project implements a **Customer Support Assistant** that:

- Reads a PDF knowledge base
- Answers customer queries
- Escalates complex queries to humans

---

2. System Architecture Explanation

The system follows a pipeline architecture:

```
User Query
    ↓
Embedding Generation
    ↓
Vector Database Search
    ↓
Retrieve Relevant Chunks
    ↓
LLM Response Generation
    ↓
Output / HITL
```

---

3. Design Decisions

3.1 Chunk Size Selection

- Chunk size: ~300
- Overlap: ~50

Reason:
Large chunks improve context but slow down processing, while smaller chunks are faster but may lose context. A balanced size is chosen.

3.2 Embedding Strategy

- Semantic embeddings are used
- Text is converted into vector format

Benefit:
Enables meaning-based search instead of keyword matching.

3.3 Retrieval Approach

- Similarity search using vector database
- Retrieves most relevant chunks

3.4 Prompt Design Logic

- Uses retrieved chunks as context
- Ensures answers are based on document

4. Workflow Explanation (LangGraph)

```
+-------------+
| Input Node  |
+------+------+
       |
       v
+-------------+
| Processing  |
| Node        |
+------+------+
       |
       v
+-------------+
| Output Node |
+-------------+
```

Explanation:

- Input Node receives query
- Processing Node retrieves and generates answer
- Output Node displays result

---

5. Conditional Logic

```
         Retrieved Answer
                  |
     +----------+----------+
     |                     |
Relevant              Not Relevant
     |                     |
     v                     v
Show Answer          Escalate to HITL
```

Explanation

The system evaluates:

- Relevance of retrieved data
- Confidence level

If valid → answer is returned
If invalid → escalation is triggered

---

6. HITL (Human-in-the-Loop) Implementation

```
User Query
    ↓
```

```
System Processing
    ↓
Low Confidence / No Answer
    ↓
Send to Human
    ↓
Human Response
    ↓
Return to User
```

Role of HITL

- Acts as fallback mechanism
- Ensures accuracy
- Handles complex queries

Benefits

- Improves reliability
- Builds user trust

Limitations

- Requires human support
- Slower response time

---

7. Challenges & Trade-offs

7.1 Retrieval Accuracy vs Speed

- High accuracy → slower
- Faster response → less accurate

7.2 Chunk Size vs Context Quality

- Larger chunks → better understanding
- Smaller chunks → faster processing

7.3 Cost vs Performance

- High-performance models → costly
- Lightweight models → less accurate

---

8. Testing Strategy

Testing Approach

- Use sample queries
- Validate responses

Sample Queries

- What is refund policy?
- How to return a product?
- What is delivery time?

Evaluation Criteria

- Accuracy
- Relevance
- Response time

---

9. Future Enhancements

- Multi-document support
- Chat memory integration
- Feedback learning system
- Web-based deployment
- Advanced LLM integration

---

10. Conclusion

This project demonstrates the design of a **RAG-based Customer Support Assistant** that:

- Uses document-based knowledge
- Retrieves relevant information efficiently
- Generates accurate responses
- Applies workflow-based decision logic

- Supports human escalation

The system reflects a real-world scalable AI solution for customer support.